

REPUBLIQUE DU SENEGAL



Un peuple – Un but – Une fois

MINISTRE DE L' ENSEIGNEMENT SUPERIEUR DE LA RECHERCHE

ET DE L' INNOVATION

UNIVERSITE ASSANE SECK DE ZIGUINCHOR

FACULTE DES SCIENCES ET TECHNOLOGIES



PROJET DE MACHINE LEARNING

SUJET 3 : Prédiction du diabète à l'aide du Machine Learning

Application au jeu de données Pima Indians Diabetes Database

Réalisée par :

Mouhamed sene

Encadré par :

Dr. Yoro Dia

Promotion 2023

REMERCIEMENTS

Je tiens à remercier toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce projet.

Je remercie particulièrement mon encadrant le **Dr. Yoro Dia** pour ses conseils, son accompagnement et ses orientations tout au long de ce travail.

Je remercie également mon établissement et l'ensemble des enseignants pour les connaissances et les compétences acquises durant ma formation.

Table des matières

INTRODUCTION	4
1. PRÉSENTATION DU PROBLÈME	4
1.1 Contexte.....	4
1.2 Problématique.....	4
1.3 Objectifs	5
2. PRÉSENTATION ET PRÉPARATION DES DONNÉES.....	5
2.1 Source du dataset.....	5
2.2 Description des variables.....	5
2.3 Exploration des données.....	6
2.4 Nettoyage des données	8
3. Méthodologie et modèles	9
3.1 Préparation des données	9
3.2 Séparation des données	11
3.3 Régression logistique	12
3.4 Arbre de décision.....	13
3.5 Random Forest	14
4. Résultats et évaluation.....	14
4.1 Résultats des modèles.....	15
4.2 Matrices de confusion	15

4.3 Mesures de performance	18
4.3.1 Accuracy.....	19
4.3.2 Precision	19
4.3.3 Recall.....	20
4.3.4 F1-score.....	20
4.3.5 Résumé des mesures.....	21
4.4 Comparaison des modèles	21
4.5 Résultats AUC de chaque modele	23
4.6 Courbe ROC de chaque modele.....	24
5. Analyse et discussion	25
5.1 Analyse générale des résultats.....	25
5.2 Analyse des matrices de confusion.....	25
5.3 Comparaison des modèles	26
5.4 Importance des variables	26
5.5 Limites de l'étude.....	28
5.6 Perspectives d'amélioration	28
6 . Conclusion.....	29
7. Bibliographie	30
8. Annexes	30
Annexe A – Code Python.....	30
A.1 Importation des bibliothèques	30
A.2 Chargement et exploration des données.....	31
A.3 Identification des valeurs nulles	31
A.4 Nettoyage des données.....	31
A.5 Préparation de X et y.....	31
A.6 Séparation entraînement/test.....	31

INTRODUCTION

Le diabète est une maladie qui représente un problème important de santé. Les nouvelles technologies, notamment l'intelligence artificielle et le Machine Learning, permettent d'utiliser les données médicales pour réaliser des prédictions.

Dans le cadre de ce projet, nous allons étudier la possibilité de prédire la présence du diabète à partir de différentes caractéristiques médicales. Pour cela, nous utiliserons le jeu de données **Pima Indians Diabetes Database**.

Les données seront étudiées et préparées à l'aide de Python et de plusieurs bibliothèques de Machine Learning. Trois modèles de classification seront utilisés : **la régression logistique, l'arbre de décision et la Random Forest**.

Les résultats obtenus seront ensuite évalués et comparés à l'aide de différentes mesures de performance.

L'objectif de ce projet est ainsi de mettre en pratique les principales étapes d'un projet de Machine Learning, depuis la préparation des données jusqu'à l'évaluation des modèles.

1. PRÉSENTATION DU PROBLÈME

1.1 Contexte

Le Machine Learning est une branche de l'intelligence artificielle qui permet à un ordinateur d'apprendre à partir de données afin de réaliser des prédictions.

Dans ce projet, nous appliquons le Machine Learning à la **prédiction du diabète**. Le modèle utilise plusieurs informations médicales concernant une personne afin de prédire si elle appartient à la catégorie des personnes diabétiques ou non diabétiques.

Cette étude nous permet de mettre en pratique les techniques de classification sur un jeu de données réel.

1.2 Problématique

La problématique de ce projet est la suivante :

Comment utiliser les techniques de Machine Learning pour prédire la présence du diabète à partir de caractéristiques médicales ?

1.3 Objectifs

L'objectif principal est de construire un modèle capable de prédire la présence ou l'absence du diabète à partir de plusieurs caractéristiques médicales.

Pour atteindre cet objectif, nous allons :

- explorer les données ;
- identifier les problèmes éventuels dans les données ;
- nettoyer et préparer le dataset ;
- utiliser plusieurs modèles de classification ;
- évaluer leurs performances ;
- comparer les résultats obtenus.

2. PRÉSENTATION ET PRÉPARATION DES DONNÉES

2.1 Source du dataset

Pour réaliser ce projet, nous avons utilisé le **Pima Indians Diabetes Database**
raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv

.

Ce jeu de données contient **768 observations et 9 colonnes**, dont huit variables explicatives et une variable cible appelée **Outcome**.

La variable **Outcome** représente le résultat à prédire :

- **0 : absence de diabète**
- **1 : présence de diabète**

Le dataset a été chargé dans Python à l'aide de la bibliothèque **Pandas**.

2.2 Description des variables

Variable	Description
Pregnancies	Nombre de grossesses
Glucose	Niveau de glucose
BloodPressure	Pression artérielle
SkinThickness	Épaisseur de la peau
Insulin	Niveau d'insuline
BMI	Indice de masse corporelle
DiabetesPedigreeFunction	Fonction liée aux antécédents familiaux de diabète
Age	Âge
Outcome	Résultat : diabète ou absence de diabète

2.3 Exploration des données

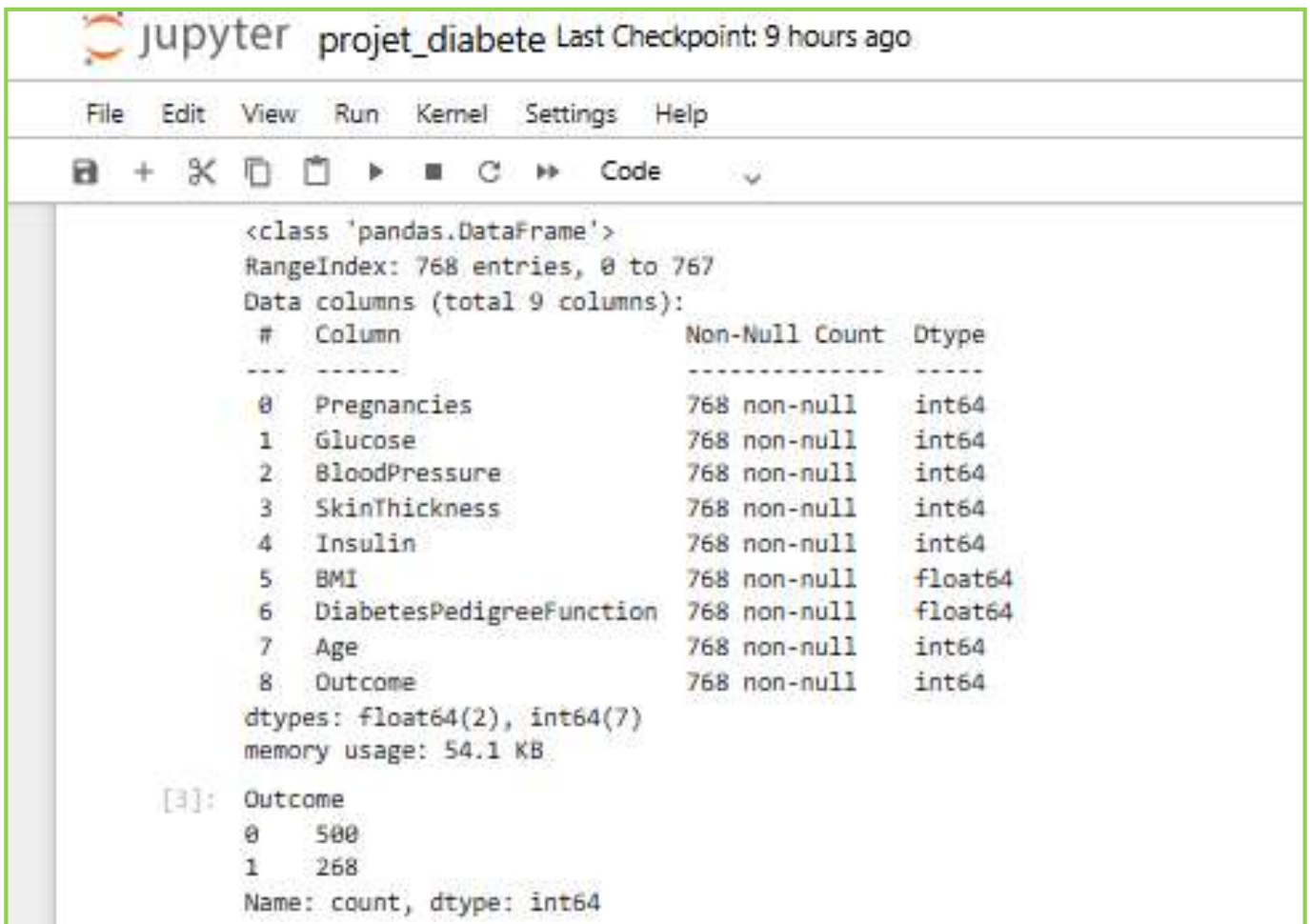
L'exploration du dataset a permis de vérifier sa structure, ses types de données et ses principales statistiques.

Le dataset contient **768 observations et 9 variables**. Il ne contient initialement aucune valeur NaN. Cependant, certaines variables médicales contiennent des valeurs égales à zéro qui peuvent correspondre à des données non renseignées.

La variable **Outcome** contient :

- 500 observations de classe 0
- 268 observations de classe 1

Capture 1 – Répartition des classes de la variable Outcome



```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Pregnancies           768 non-null   int64  
1   Glucose               768 non-null   int64  
2   BloodPressure         768 non-null   int64  
3   SkinThickness         768 non-null   int64  
4   Insulin               768 non-null   int64  
5   BMI                   768 non-null   float64 
6   DiabetesPedigreeFunction 768 non-null   float64 
7   Age                   768 non-null   int64  
8   Outcome               768 non-null   int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB

[3]: Outcome
0    500
1    268
Name: count, dtype: int64
```

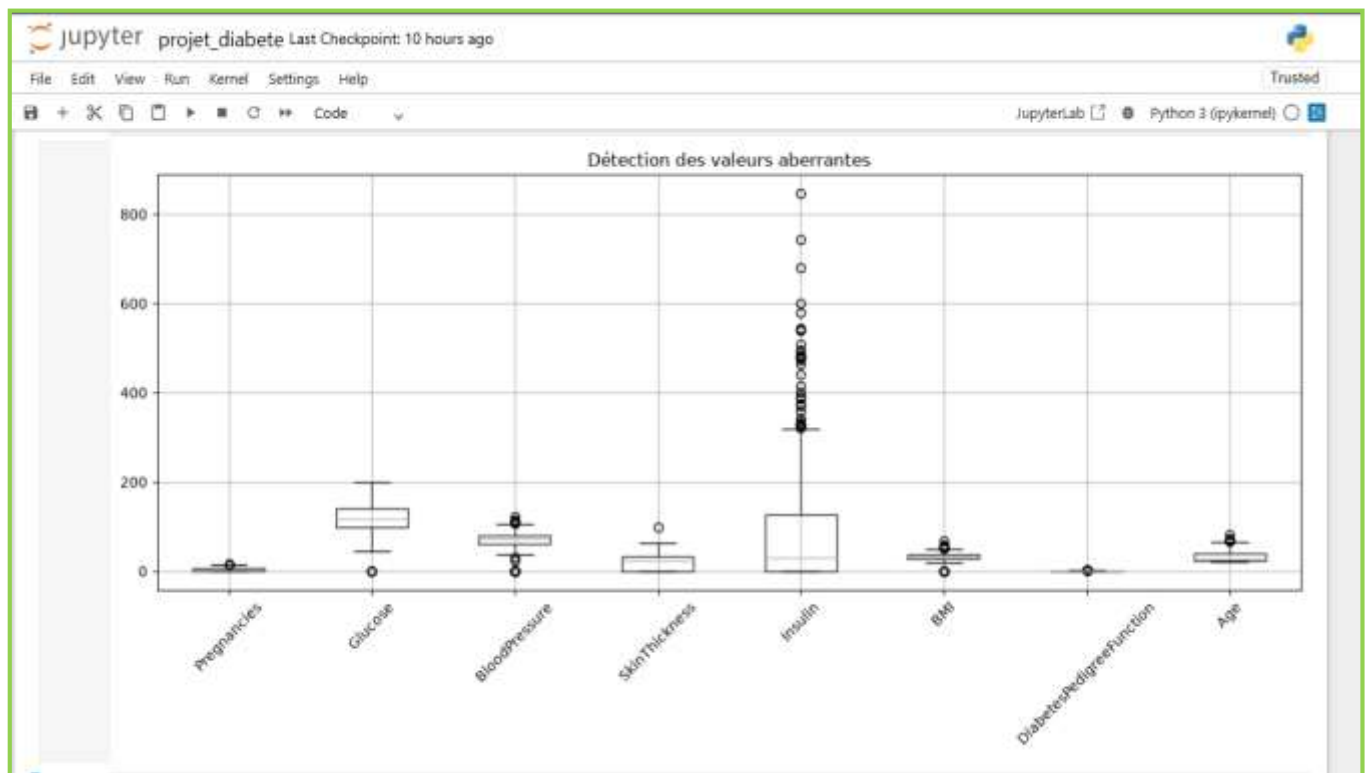
Capture 2 – Histogrammes des variables



2.4 Nettoyage des données

Les boxplots permettent d'observer la dispersion des variables ainsi que certaines valeurs extrêmes. La variable Insulin présente notamment une dispersion importante. Ces valeurs n'ont pas été supprimées automatiquement, car elles peuvent représenter de véritables observations.

Capture 3 – Boxplots des variables explicatives avant nettoyage



Lors de l'analyse, nous avons constaté la présence de valeurs nulles dans plusieurs variables médicales.

Les variables concernées sont :

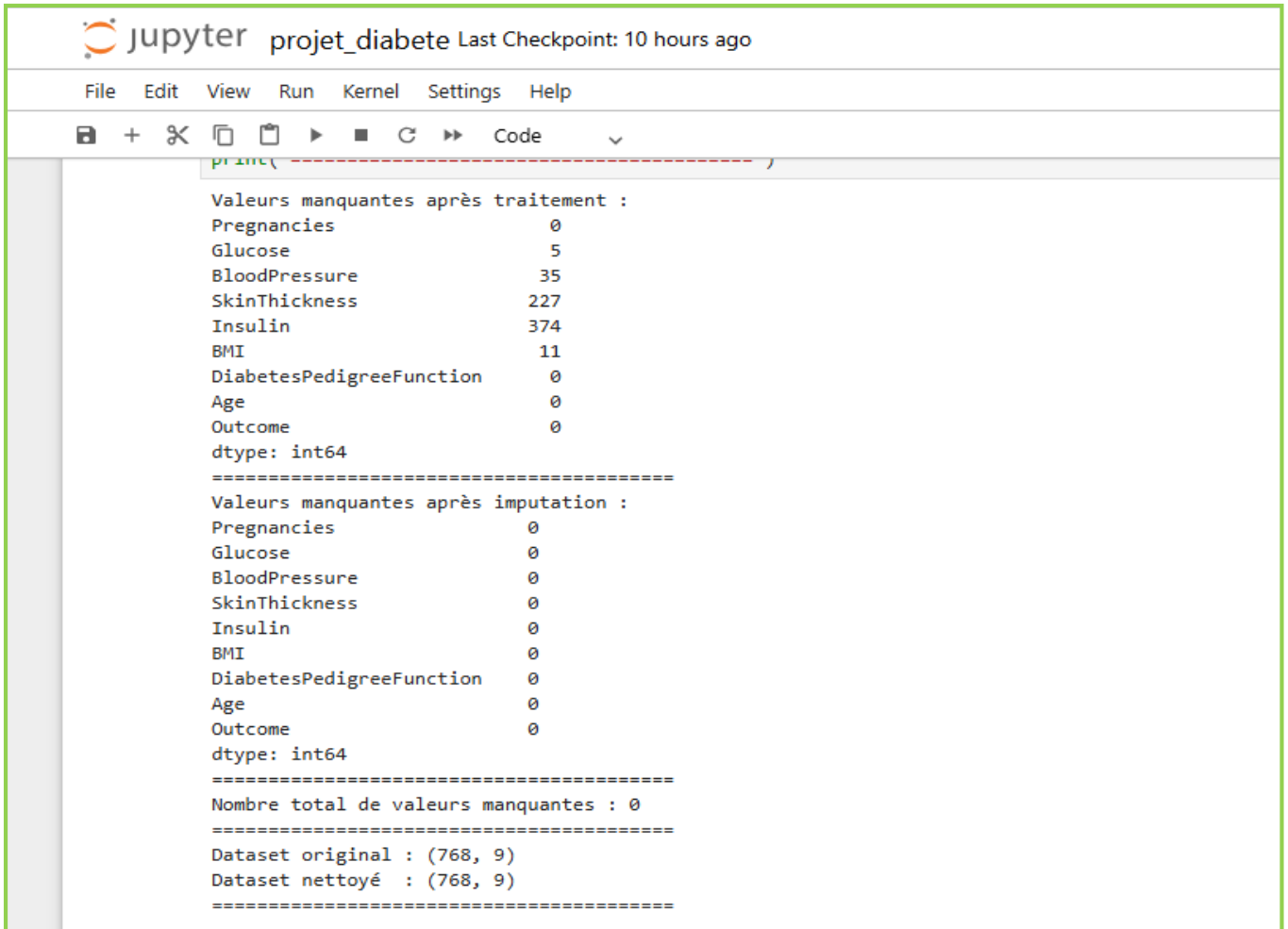
- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI

Ces valeurs ont été considérées comme des valeurs manquantes.

Elles ont ensuite été remplacées par **la médiane de chaque variable**. Cette méthode permet de conserver les observations tout en limitant l'influence des valeurs extrêmes.

Après cette opération, le dataset ne contient plus de valeurs manquantes et conserve ses **768 observations**.

Capture 4 – Apres nettoyage des donnees



```
print(
    Valeurs manquantes après traitement :
    Pregnancies      0
    Glucose          5
    BloodPressure    35
    SkinThickness    227
    Insulin          374
    BMI              11
    DiabetesPedigreeFunction  0
    Age              0
    Outcome          0
    dtype: int64
    =====
    Valeurs manquantes après imputation :
    Pregnancies      0
    Glucose          0
    BloodPressure    0
    SkinThickness    0
    Insulin          0
    BMI              0
    DiabetesPedigreeFunction  0
    Age              0
    Outcome          0
    dtype: int64
    =====
    Nombre total de valeurs manquantes : 0
    =====
    Dataset original : (768, 9)
    Dataset nettoyé   : (768, 9)
    =====
```

3. Méthodologie et modèles

Cette partie présente les différentes étapes utilisées pour construire les modèles de Machine Learning. Après le nettoyage des données, nous avons préparé les variables, séparé les données en ensembles d'entraînement et de test, puis entraîné trois modèles de classification : la régression logistique, l'arbre de décision et la Random Forest.

3.1 Préparation des données

Après le nettoyage des données, nous avons séparé le dataset en deux parties : les variables explicatives et la variable cible.

La variable **Outcome** représente la variable à prédire :

- **0** : absence de diabète ;
- **1** : présence de diabète.

Les huit autres variables sont utilisées comme variables explicatives :

- Pregnancies ;
- Glucose ;
- BloodPressure ;
- SkinThickness ;
- Insulin ;
- BMI ;
- DiabetesPedigreeFunction ;
- Age.

Ainsi, la matrice **X** contient les huit variables explicatives et le vecteur **Y** contient la variable cible Outcome.

Après cette séparation, nous obtenons :

- **X : (768, 8) ;**
- **y : (768,).**

Cela signifie que nous disposons de 768 observations et de 8 variables utilisées pour effectuer les prédictions.

Capture 5 – Séparation des variables explicatives et de la variable cible

```
51]: print("Variables explicatives :")
print(X.columns.tolist())
print("\nVariable cible :")
print(y.name)
# Variables explicatives
X = df_clean.drop("Outcome", axis=1)

# Variable cible
y = df_clean["Outcome"]
print("=====")
print("Dimensions de X :", X.shape)
print("Dimensions de y :", y.shape)
```

Variables explicatives :

['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

Variable cible :

Outcome

=====

Dimensions de X : (768, 8)

Dimensions de y : (768,)

3.2 Séparation des données

Afin d'évaluer les modèles sur des données qu'ils n'ont pas utilisées pendant leur apprentissage, le dataset a été séparé en deux ensembles :

- **80 % des données pour l'entraînement ;**
- **20 % des données pour le test.**

Capture 6 – Répartition des données entre entraînement et test



```
[24]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train :", X_train.shape)
print("X_test  :", X_test.shape)
print("y_train :", y_train.shape)
print("y_test  :", y_test.shape)

X_train : (614, 8)
X_test  : (154, 8)
y_train : (614,)
y_test  : (154,)
```

La fonction `train_test_split` de Scikit-learn a été utilisée avec `random_state=42` afin de conserver une séparation reproductible. L'option `stratify=y` permet de conserver une répartition similaire des classes 0 et 1 dans les ensembles d'entraînement et de test.

La séparation obtenue est la suivante :

- **X_train : 614×8 ;**
- **X_test : 154×8 ;**
- **y_train : 614 ;**

- y_{test} : 154.

Les modèles sont donc entraînés sur les 614 observations de l'ensemble d'entraînement, puis évalués sur les 154 observations de l'ensemble de test.

3.3 Régression logistique

La régression logistique est un modèle de classification utilisé ici pour déterminer si une personne appartient à la classe **0 (non diabétique)** ou à la classe **1 (diabétique)**.

Contrairement à une régression linéaire classique, la régression logistique permet d'obtenir une probabilité comprise entre 0 et 1 grâce à la **fonction sigmoïde**.

Principe mathématique

Le modèle commence par calculer une combinaison linéaire des variables :

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

où :

- $X_1, X_2, \dots, X_n$ représentent les variables explicatives ;
- β_0 représente la constante du modèle ;
- $\beta_1, \beta_2, \dots, \beta_n$ représentent les coefficients associés aux variables.

Cette valeur est ensuite introduite dans la fonction sigmoïde :

$$P(Y = 1 | X) = \frac{1}{1 + e^{-z}}$$

Cette fonction permet d'obtenir une probabilité comprise entre 0 et 1.

Dans notre problème :

- si la probabilité est suffisamment élevée, le modèle prédit **1**, c'est-à-dire présence du diabète ;
- sinon, il prédit **0**, c'est-à-dire absence de diabète.

La régression logistique a été entraînée avec le code suivant :

Capture 7– Entraînement du modèle de régression logistique

```
[26]: from sklearn.linear_model import LogisticRegression

# Création du modèle
model_lr = LogisticRegression(max_iter=1000, random_state=42)

# Entraînement du modèle
model_lr.fit(X_train, y_train)

print("Modèle de régression logistique entraîné avec succès.")
```

Modèle de régression logistique entraîné avec succès.

3.4 Arbre de décision

L'arbre de décision est un modèle de classification qui prend des décisions successives à partir des variables disponibles.

Le modèle fonctionne sous la forme d'une succession de questions ou de règles. Par exemple, il peut utiliser une variable comme le niveau de glucose pour séparer les observations en différents groupes. D'autres variables peuvent ensuite être utilisées pour poursuivre la séparation.

L'objectif est d'arriver à des groupes permettant de distinguer les personnes diabétiques des personnes non diabétiques.

Dans notre projet, l'arbre de décision a été entraîné avec le code suivant :

Capture 8 – Entraînement du modèle d'arbre de décision

```
[33]: from sklearn.tree import DecisionTreeClassifier

model_dt = DecisionTreeClassifier(random_state=42)

model_dt.fit(X_train, y_train)

print("Arbre de décision entraîné avec succès.")
```

Arbre de décision entraîné avec succès.

3.5 Random Forest

La Random Forest est une méthode qui combine plusieurs arbres de décision afin d'obtenir une prédiction finale.

Au lieu de dépendre d'un seul arbre, le modèle construit plusieurs arbres et combine leurs résultats. Cette approche permet d'obtenir un modèle plus robuste que l'utilisation d'un arbre unique.

Dans notre projet, nous avons utilisé **100 arbres** avec `random_state=42`.

Le modèle a été entraîné avec le code suivant :

Capture 9– Entraînement du modèle Random Forest

```
[40]: from sklearn.ensemble import RandomForestClassifier

model_rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model_rf.fit(X_train, y_train)

print("Random Forest entraînée avec succès.")

Random Forest entraînée avec succès.
```

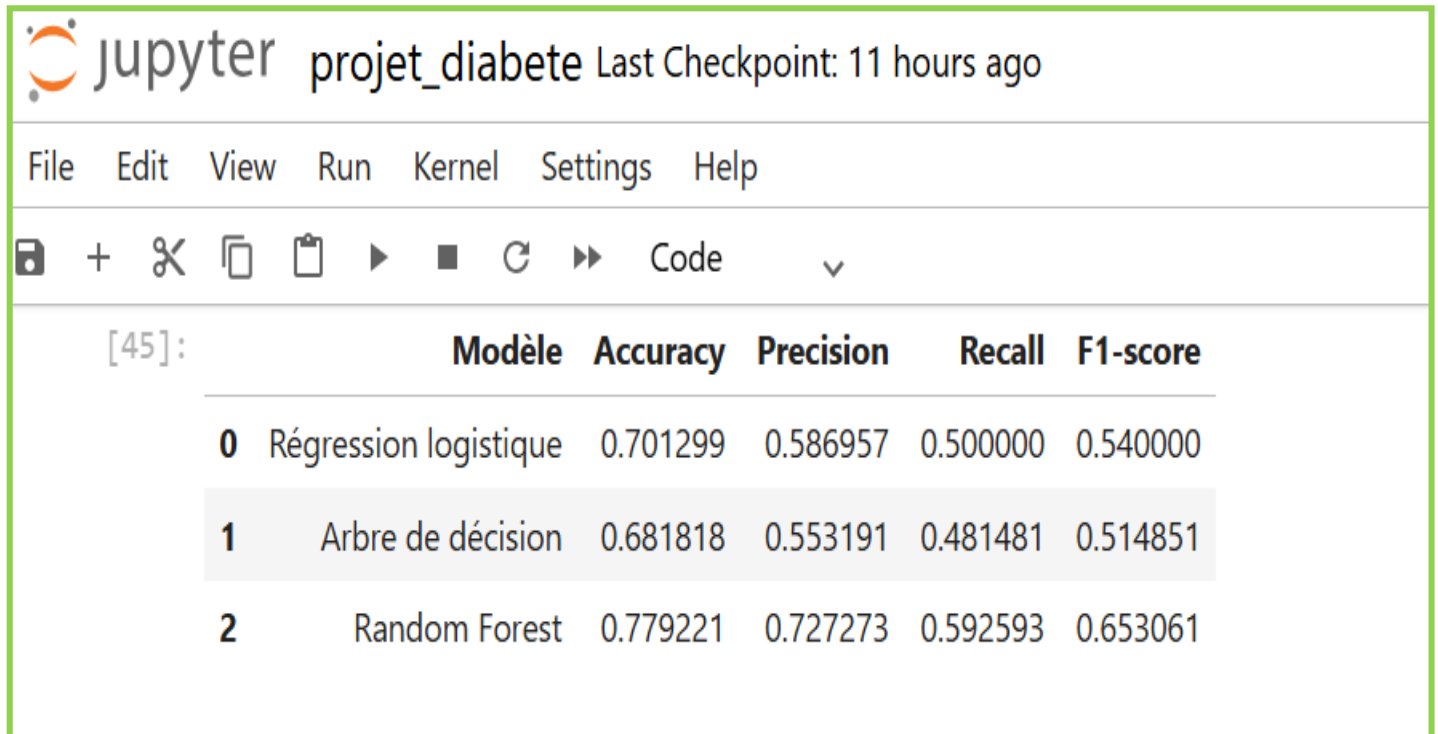
4. Résultats et évaluation

Après l'entraînement des trois modèles, nous avons effectué des prédictions sur les données de test. Les modèles ont ensuite été évalués à l'aide de plusieurs mesures : **Accuracy**, **Precision**, **Recall** et **F1-score**.

4.1 Résultats des modèles

Les résultats obtenus sur l'ensemble de test sont présentés dans le tableau suivant :

Capture 10 – Valeurs numériques des performances des trois modèles



The screenshot shows a Jupyter Notebook window titled 'projet_diabete' with a 'Last Checkpoint: 11 hours ago'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for saving, adding, deleting, copying, pasting, and running code. Below the toolbar, a code cell contains a table of performance metrics for three models. The table has columns for 'Modèle', 'Accuracy', 'Precision', 'Recall', and 'F1-score'. The rows represent three models: 'Régression logistique', 'Arbre de décision', and 'Random Forest'.

[45]:	Modèle	Accuracy	Precision	Recall	F1-score
0	Régression logistique	0.701299	0.586957	0.500000	0.540000
1	Arbre de décision	0.681818	0.553191	0.481481	0.514851
2	Random Forest	0.779221	0.727273	0.592593	0.653061

Ces résultats permettent de comparer les performances des trois modèles sur les mêmes données de test.

4.2 Matrices de confusion

Pour mieux comprendre les résultats obtenus, nous avons utilisé les **matrices de confusion**.

Une matrice de confusion permet de comparer les valeurs réelles avec les valeurs prédites par le modèle.

Elle contient quatre éléments importants :

Prédiction	Réalité : 0	Réalité : 1
Prédiction : 0	TN	FN
Prédiction : 1	FP	TP

Signification des termes

TN – True Negative : le modèle prédit 0 et la valeur réelle est 0. La prédiction est correcte.

TP – True Positive : le modèle prédit 1 et la valeur réelle est 1. La prédiction est correcte.

FP – False Positive : le modèle prédit 1 alors que la valeur réelle est 0. Il s'agit d'une fausse alerte.

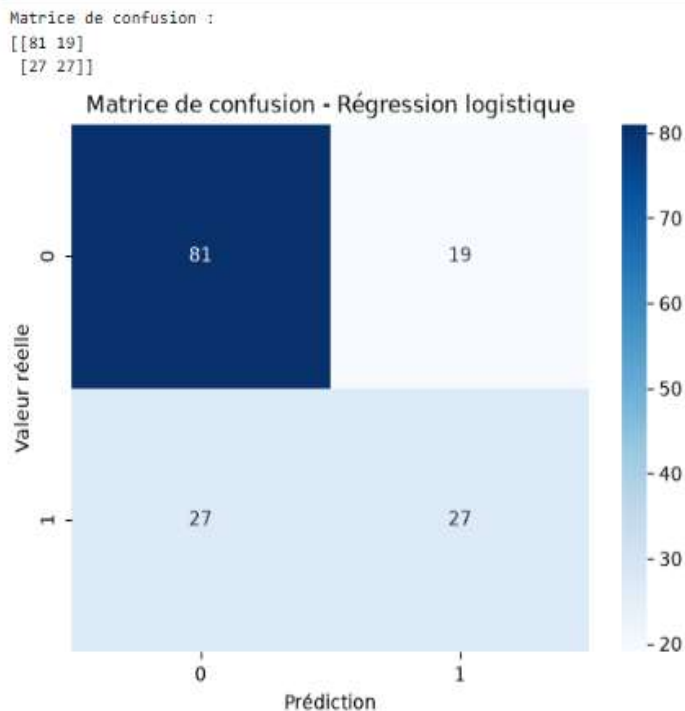
FN – False Negative : le modèle prédit 0 alors que la valeur réelle est 1. Le modèle n'a donc pas détecté une personne réellement diabétique.

Dans notre problème, les **FN sont particulièrement importants à observer**, car ils correspondent aux cas de diabète que le modèle n'a pas détectés.

📊 Régression logistique

La matrice de confusion obtenue est :

Capture 11 – Matrice de confusion de la régression logistique



On obtient donc :

- **TN = 81**
- **FP = 19**
- **FN = 27**
- **TP = 27**

Le modèle a correctement classé 81 personnes non diabétiques et 27 personnes diabétiques.

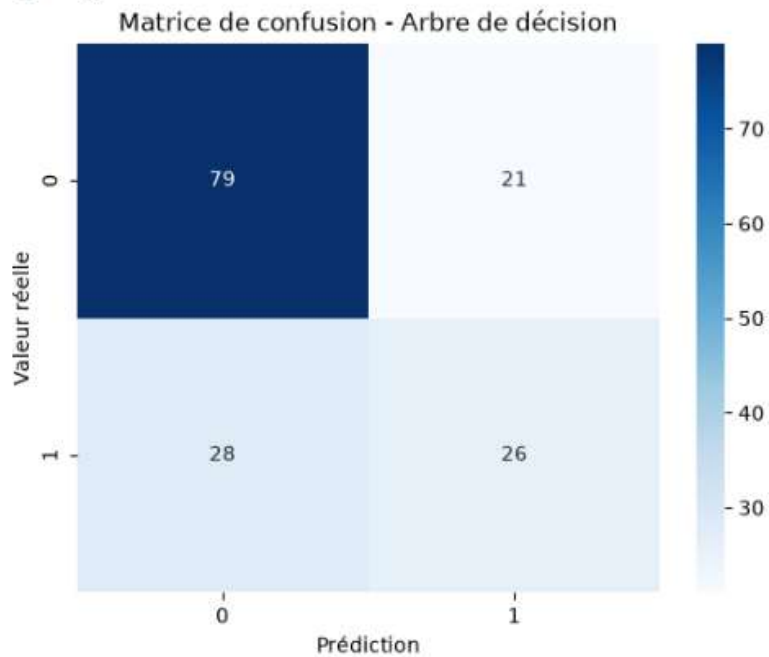
🚦 Arbre de décision

La matrice obtenue est :

Capture 12 – Matrice de confusion de l'arbre de décision

Matrice de confusion :

```
[[79 21]  
 [28 26]]
```



On obtient :

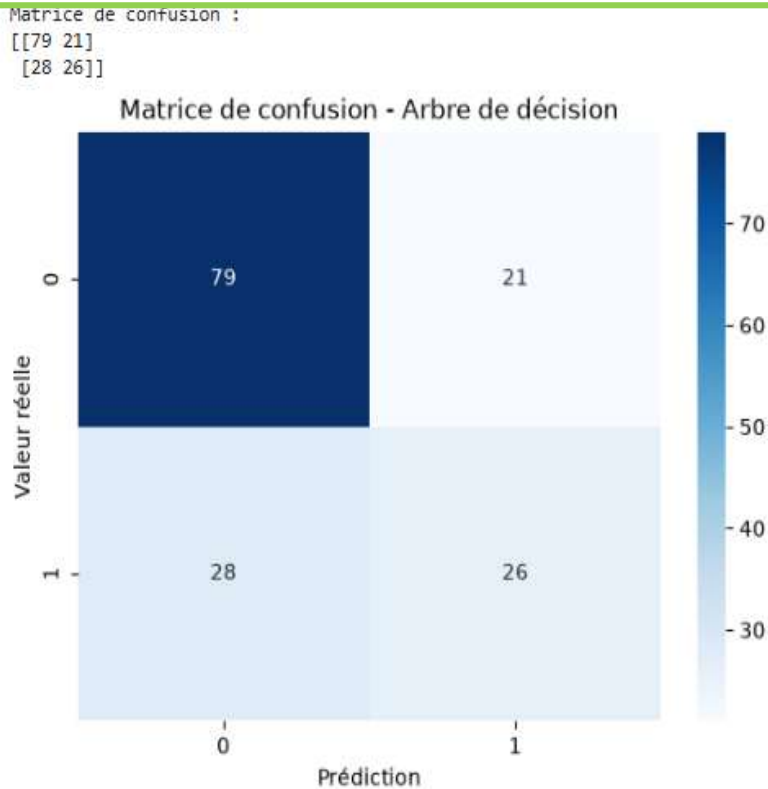
- **TN = 79**
- **FP = 21**
- **FN = 28**
- **TP = 26**

Le modèle a correctement classé 79 personnes non diabétiques et 28 personnes diabétiques.

Random Forest

La matrice obtenue est :

Capture 13 – Matrice de confusion de la Random Forest



On obtient :

- **TN = 88**
- **FP = 12**
- **FN = 22**
- **TP = 32**

La Random Forest a donc correctement classé 88 personnes non diabétiques et 32 personnes diabétiques.

4.3 Mesures de performance

Afin d'évaluer les modèles, quatre mesures principales ont été utilisées : **Accuracy**, **Precision**, **Recall** et **F1-score**.

Ces mesures sont calculées à partir des quatre éléments de la matrice de confusion : **TP**, **TN**, **FP** et **FN**.

4.3.1 Accuracy

L'Accuracy mesure la proportion totale de prédictions correctes.

La formule est :

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

Pour la Random Forest, nous avons :

- TP = 32
- TN = 88
- FP = 12
- FN = 22

Donc :

$$Accuracy = \frac{32 + 88}{32 + 88 + 12 + 22} \approx 0,7792$$

Soit :

$$Accuracy = 77,92\%$$

4.3.2 Precision

La Precision mesure la proportion de prédictions positives qui sont réellement positives.

Autrement dit, parmi les personnes que le modèle a prédites comme diabétiques, elle indique combien sont réellement diabétiques.

La formule est :

$$Precision = \frac{TP}{TP + FP}$$

Pour la Random Forest :

$$Precision = \frac{32}{32 + 12} \approx 0,7273$$

Soit :

$$Precision = 72,73\%$$

4.3.3 Recall

Le Recall, également appelé **sensibilité**, mesure la capacité du modèle à détecter les personnes réellement positives.

Dans notre projet, il indique donc la capacité du modèle à détecter les personnes réellement diabétiques.

La formule est :

$$Recall = \frac{TP}{TP + FN}$$

Pour la Random Forest :

$$Recall = \frac{32}{32+22} \approx 0,5926$$

Soit :

$$Recall = 59,26\%$$

4.3.4 F1-score

Le F1-score combine la Precision et le Recall dans une seule mesure.

Sa formule est :

$$F1 = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)}$$

Pour la Random Forest :

$$F1 = 2 \times \frac{(0,7273 \times 0,5926)}{(0,7273 + 0,5926)} \approx 0,6531$$

Soit :

$$F1 = 65,31\%$$

Le F1-score est donc utile lorsque l'on souhaite prendre en compte simultanément la précision des prédictions positives et la capacité du modèle à détecter les cas positifs.

4.3.5 Résumé des mesures

Les quatre mesures peuvent être résumées ainsi :

Mesure	Formule	Signification
Accuracy	$\frac{(TP + TN)}{(TP + TN + FP + FN)}$	Proportion de prédictions correctes
Precision	$\frac{(TP)}{(TP + FP)}$	Fiabilité des prédictions positives
Recall	$\frac{(TP)}{(TP + FN)}$	Capacité à détecter les vrais positifs
F1-score	$2 \times \frac{(Precision \times Recall)}{(Precision + Recall)}$	Équilibre entre Precision et Recall

Cette présentation permet de relier directement les résultats obtenus avec les prédictions du modèle et les éléments de la matrice de confusion.

4.4 Comparaison des modèles

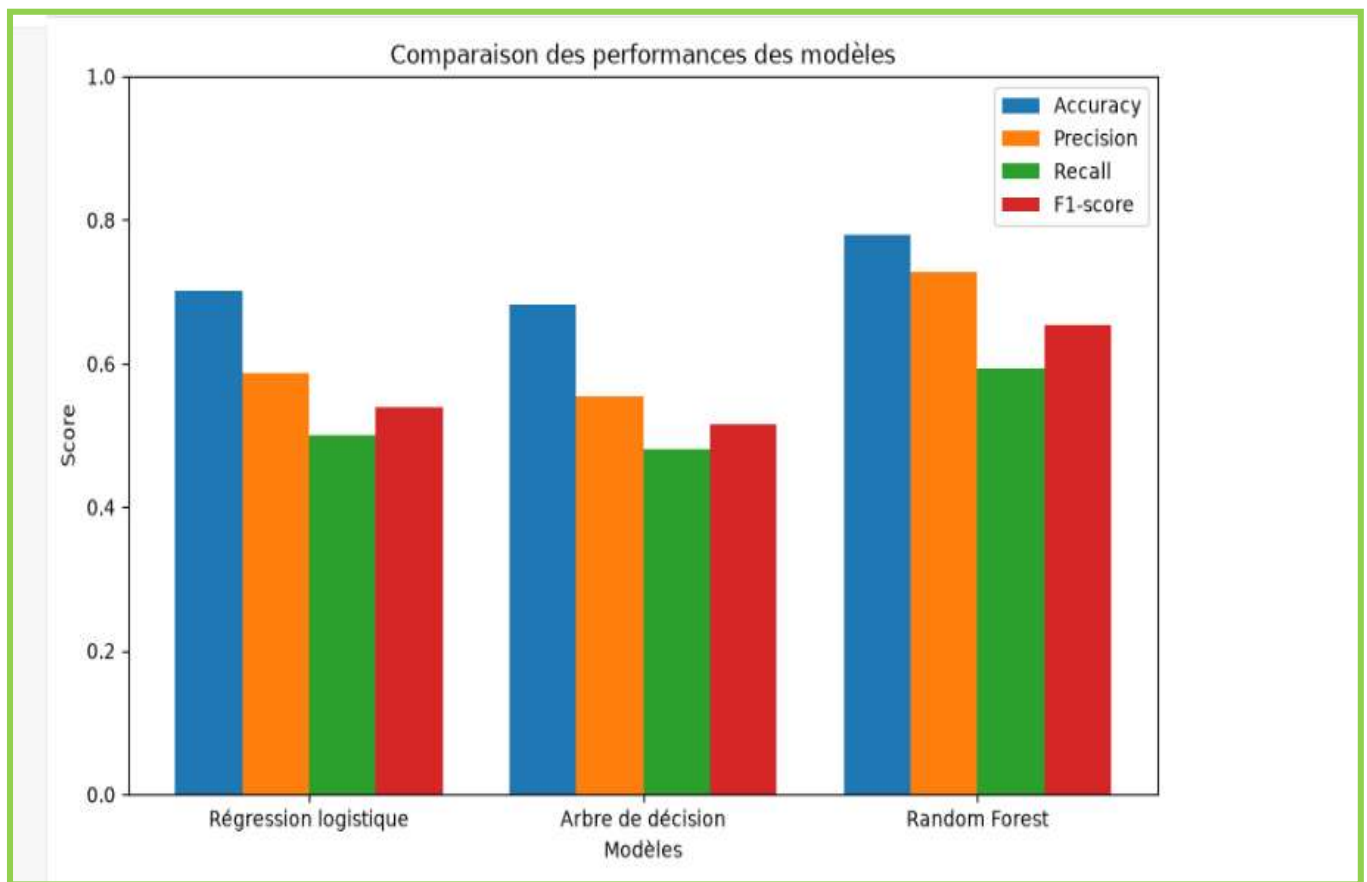
La comparaison des trois modèles permet d'observer leurs performances sur l'ensemble de test, le graphique suivant permet de visualiser cette comparaison.

La **régression logistique** obtient une Accuracy de **70,13 %**, une Precision de **58,70 %**, un Recall de **50,00 %** et un F1-score de **54,00 %**.

L'**arbre de décision** obtient une Accuracy de **68,18 %**, une Precision de **55,32 %**, un Recall de **48,15 %** et un F1-score de **51,49 %**.

La **Random Forest** obtient une Accuracy de **77,92 %**, une Precision de **72,73 %**, un Recall de **59,26 %** et un F1-score de **65,31 %**.

Capture 14 – Comparaison des performances des modèles



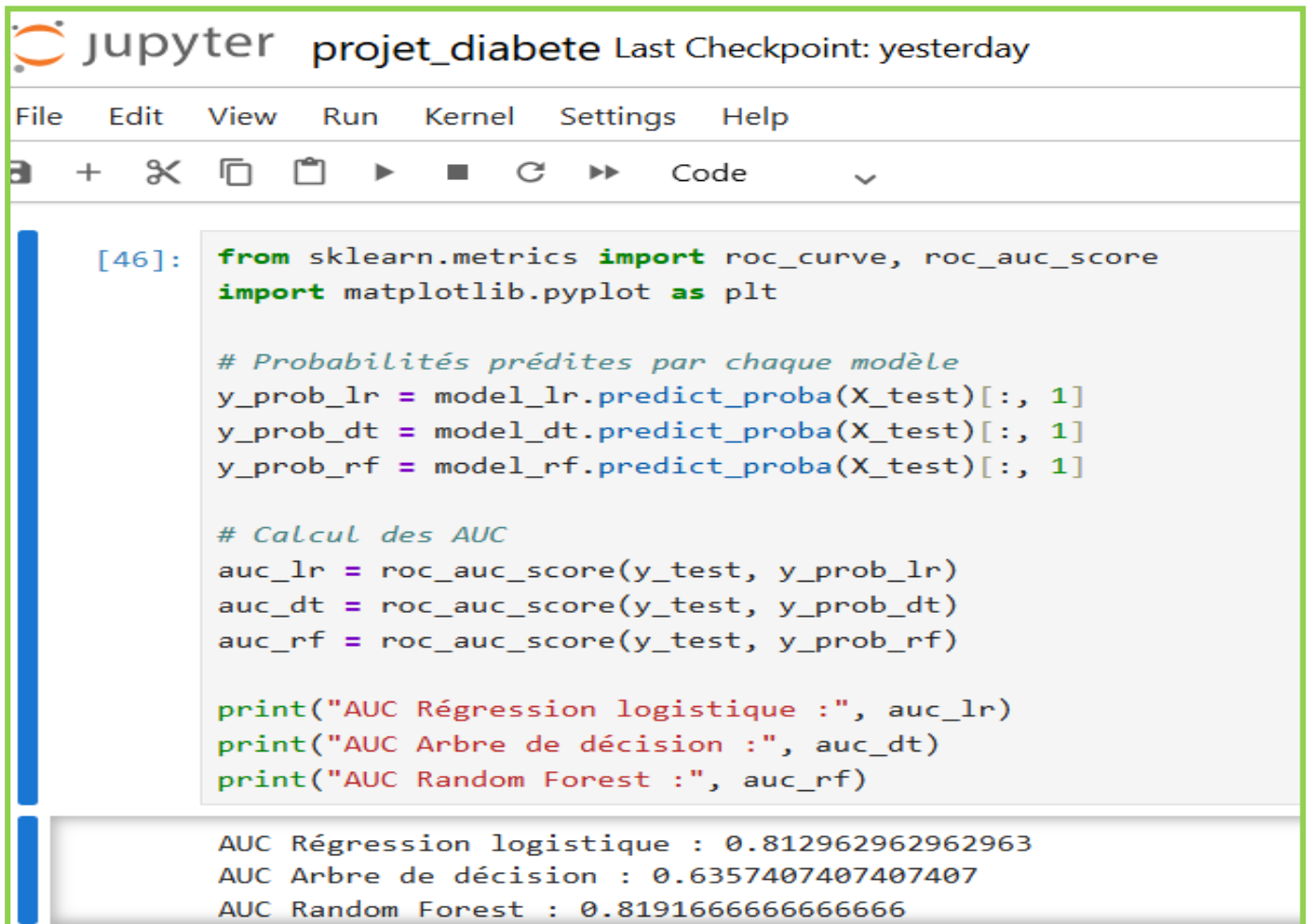
Interpretation :

Ces résultats montrent que, **sur l'ensemble de test utilisé dans notre expérience**, les valeurs mesurées sont différentes selon le modèle. La Random Forest obtient les valeurs les plus élevées pour les quatre mesures considérées dans cette expérience.

4.5 Résultats AUC de chaque modele

Les résultats obtenus sur l'ensemble des modeles sont présentés ci-dessous :

Capture 15 – AUC de chaque modele



```
[46]: from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# Probabilités prédites par chaque modèle
y_prob_lr = model_lr.predict_proba(X_test)[: , 1]
y_prob_dt = model_dt.predict_proba(X_test)[: , 1]
y_prob_rf = model_rf.predict_proba(X_test)[: , 1]

# Calcul des AUC
auc_lr = roc_auc_score(y_test, y_prob_lr)
auc_dt = roc_auc_score(y_test, y_prob_dt)
auc_rf = roc_auc_score(y_test, y_prob_rf)

print("AUC Régression logistique :", auc_lr)
print("AUC Arbre de décision :", auc_dt)
print("AUC Random Forest :", auc_rf)

AUC Régression logistique : 0.812962962962963
AUC Arbre de décision : 0.6357407407407407
AUC Random Forest : 0.8191666666666666
```

Interpretation :

Les valeurs d'AUC obtenues montrent une bonne capacité de discrimination pour la régression logistique et la Random Forest. La Random Forest obtient une AUC de 0,8192, légèrement supérieure à celle de la régression logistique (0,8130). L'arbre de décision présente une AUC plus faible, avec une valeur de 0,6357. Ces résultats sont obtenus sur l'ensemble de test utilisé dans cette étude.

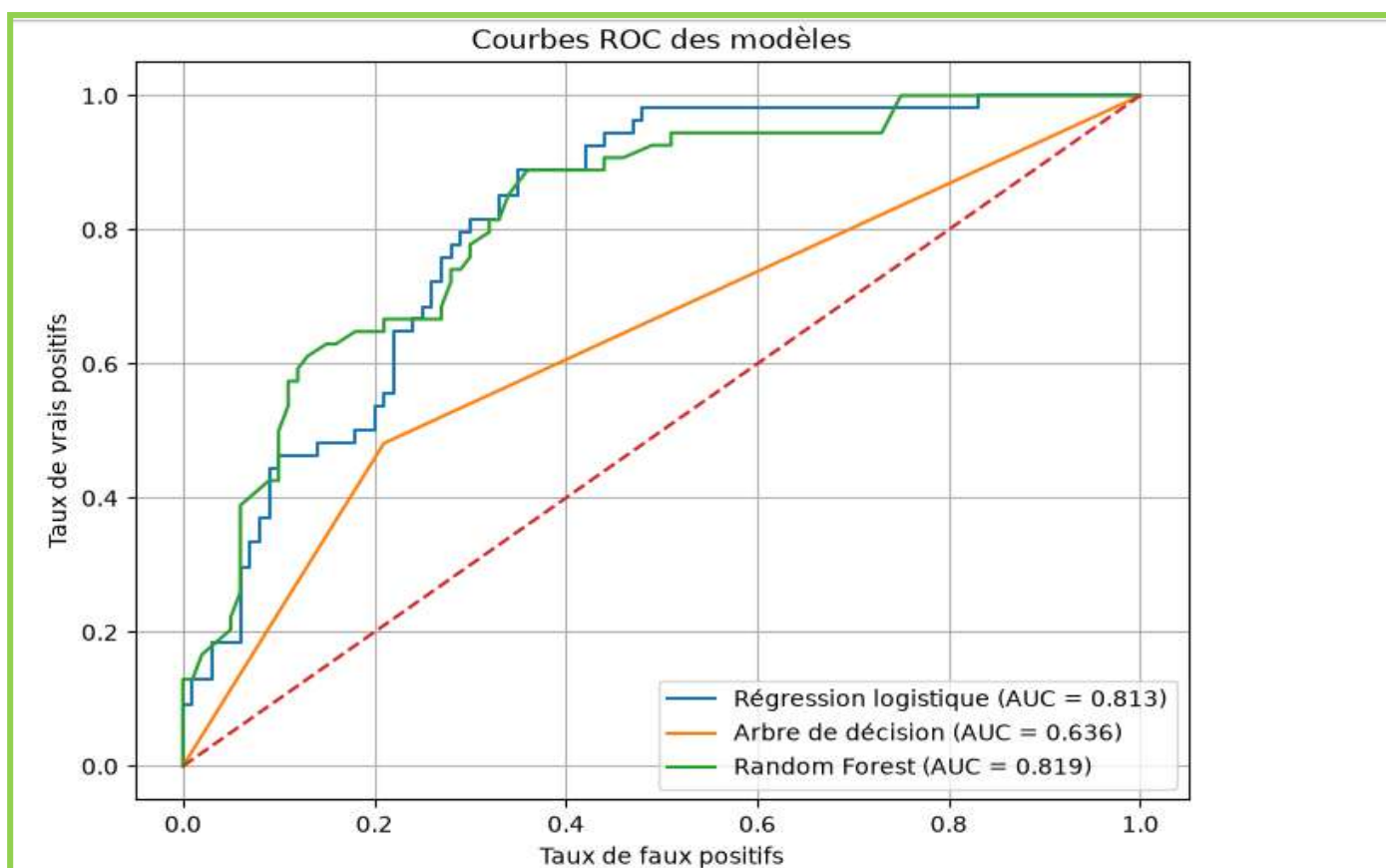
4.6 Courbe ROC de chaque modele

Les courbes ROC permettent d'évaluer la capacité des modèles à distinguer les personnes diabétiques des personnes non diabétiques. L'axe horizontal représente le taux de faux positifs, tandis que l'axe vertical représente le taux de vrais positifs.

Les résultats obtenus donnent une AUC de **0,813** pour la régression logistique, **0,636** pour l'arbre de décision et **0,819** pour la Random Forest. La Random Forest présente donc une AUC légèrement supérieure à celle de la régression logistique, tandis que l'arbre de décision présente une AUC plus faible.

La courbe de la Random Forest et celle de la régression logistique sont globalement plus éloignées de la diagonale que celle de l'arbre de décision, ce qui correspond à leurs AUC respectives de **0,819**, **0,813** et **0,636**.

Capture 16 – Courbes ROC des modèles



Interpretation :

Ces résultats montrent que les trois modèles ont une capacité de discrimination supérieure à une classification aléatoire, représentée par la diagonale. Les valeurs d'AUC sont calculées sur l'ensemble de test utilisé dans cette étude.

5. Analyse et discussion

5.1 Analyse générale des résultats

Les résultats obtenus montrent que les trois modèles étudiés présentent des performances différentes pour la prédiction du diabète.

La régression logistique obtient une Accuracy de **70,13 %** et un F1-score de **54,00 %**. L'arbre de décision obtient une Accuracy de **68,18 %** et un F1-score de **51,49 %**. La Random Forest obtient une Accuracy de **77,92 %** et un F1-score de **65,31 %**.

Dans les données de test utilisées dans cette étude, la Random Forest obtient donc les valeurs les plus élevées pour les quatre mesures étudiées : Accuracy, Precision, Recall et F1-score.

Ces résultats montrent que le choix du modèle peut avoir une influence importante sur les performances d'une tâche de classification. Cependant, les résultats obtenus dépendent également de la préparation des données et de la séparation entre les données d'entraînement et de test.

5.2 Analyse des matrices de confusion

Les matrices de confusion permettent de mieux comprendre les erreurs réalisées par les modèles.

Pour la **régression logistique**, 27 personnes diabétiques ont été correctement identifiées, tandis que 27 personnes diabétiques ont été classées à tort comme non diabétiques. Le modèle a également correctement identifié 81 personnes non diabétiques.

Pour l'**arbre de décision**, 26 personnes diabétiques ont été correctement identifiées et 28 ont été classées à tort comme non diabétiques. Le modèle a correctement classé 79 personnes non diabétiques.

Pour la **Random Forest**, 32 personnes diabétiques ont été correctement identifiées et 22 ont été classées à tort comme non diabétiques. Elle a également correctement identifié 88 personnes non diabétiques.

Ces résultats permettent de constater que les trois modèles commettent encore des erreurs de classification. En particulier, certaines personnes diabétiques sont prédites comme non diabétiques. Cette situation explique notamment pourquoi le **Recall** reste inférieur à 60 % pour les trois modèles.

5.3 Comparaison des modèles

Les résultats permettent de comparer les comportements des trois modèles.

La **régression logistique** présente une Accuracy de 70,13 %. Elle fournit donc des résultats corrects sur l'ensemble de test, mais son Recall de 50 % montre qu'une partie importante des personnes diabétiques n'est pas correctement identifiée.

L'**arbre de décision** obtient une Accuracy de 68,18 % et un F1-score de 51,49 %. Dans cette expérimentation, ses performances sont légèrement inférieures à celles de la régression logistique.

La **Random Forest** obtient une Accuracy de 77,92 %, une Precision de 72,73 %, un Recall de 59,26 % et un F1-score de 65,31 %. Sur cet ensemble de test, elle fournit donc les scores les plus élevés parmi les trois modèles étudiés.

Il faut toutefois interpréter ces résultats dans le contexte de cette expérimentation. Ils ne permettent pas à eux seuls de déterminer comment les modèles se comporteraient sur d'autres populations ou sur de nouvelles données médicales.

5.4 Importance des variables

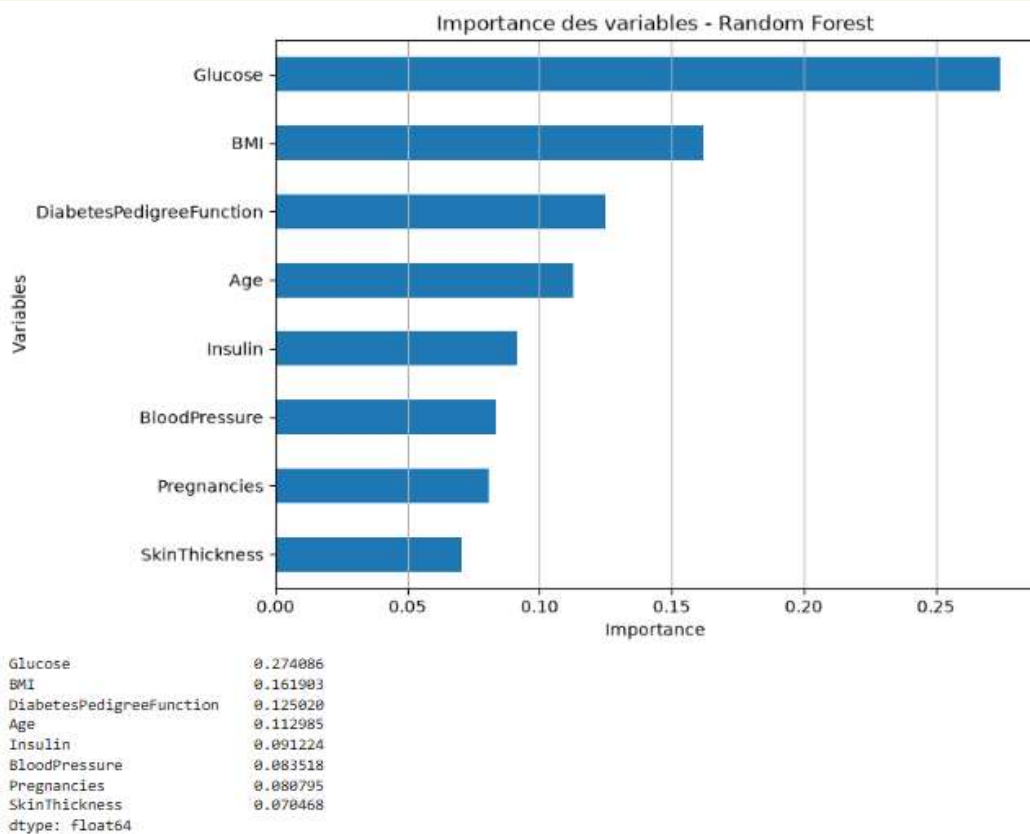
Afin de mieux interpréter le modèle Random Forest, nous avons étudié l'importance des différentes variables utilisées pour effectuer les prédictions. Les résultats montrent que **Glucose** est la variable ayant l'importance la plus élevée, avec une valeur de **0,274**, soit environ **27,4 %**. Elle est suivie par **BMI** avec **0,162 (16,2 %)** et **DiabetesPedigreeFunction** avec **0,125 (12,5 %)**.

Les variables **Age**, **Insulin**, **BloodPressure**, **Pregnancies** et **SkinThickness** présentent également une importance dans les prédictions, avec respectivement environ **11,3 %**, **9,1 %**, **8,4 %**, **8,1 %** et **7,0 %**.

Ces résultats indiquent que, parmi les variables utilisées par le modèle, le **Glucose** contribue davantage aux décisions de la Random Forest que les autres variables. Il est toutefois important de préciser que l'importance d'une variable dans un modèle ne signifie pas qu'elle est à elle seule responsable de la présence du diabète. Elle indique plutôt son rôle relatif dans les décisions effectuées par le modèle.

Le graphique présente l'importance des différentes variables utilisées par le modèle Random Forest. La variable **Glucose** est la plus importante avec une valeur de **0,274**, soit environ **27,4 %**. Elle est suivie par le **BMI** avec **16,2 %**, la **DiabetesPedigreeFunction** avec **12,5 %** et l'**Age** avec **11,3 %**. Les autres variables présentent des importances comprises entre environ 7 % et 9 %.

Capture 17 – Importance des variables dans le modèle Random Forest



Interpretation :

On peut donc constater que le **Glucose** joue un rôle particulièrement important dans les décisions du modèle. Cependant, cette importance représente la contribution relative des variables dans le fonctionnement du modèle et ne signifie pas qu'une variable est, à elle seule, responsable du diabète.

5.5 Limites de l'étude

Cette étude présente certaines limites.

Tout d'abord, le dataset utilisé contient **768 observations**, ce qui constitue un ensemble de données relativement limité pour généraliser les résultats à une population plus large.

Ensuite, certaines valeurs égales à zéro dans les variables médicales ont été considérées comme des valeurs manquantes puis remplacées par la **médiane** de chaque variable. Cette méthode permet de conserver les observations, mais elle peut également modifier légèrement la distribution originale des données.

Enfin, l'évaluation a été réalisée sur une seule séparation entre les données d'entraînement et les données de test. Les performances pourraient donc varier avec une autre séparation des données.

Ces limites doivent être prises en compte lors de l'interprétation des résultats.

5.6 Perspectives d'amélioration

Plusieurs améliorations pourraient être envisagées dans une étude future.

Il serait notamment possible d'utiliser une **validation croisée** afin d'obtenir une évaluation plus robuste des modèles. D'autres méthodes de préparation des données pourraient également être testées.

Il serait aussi possible d'ajuster les paramètres des modèles afin d'étudier leur influence sur les performances. Enfin, d'autres algorithmes de classification pourraient être testés et comparés aux trois modèles utilisés dans ce projet.

6 . Conclusion

Le présent projet avait pour objectif d'étudier l'utilisation du Machine Learning pour prédire la présence du diabète à partir de caractéristiques médicales.

Le travail a commencé par l'exploration et la préparation du **Pima Indians Diabetes Database**. Certaines valeurs nulles présentes dans les variables médicales ont été considérées comme des valeurs manquantes et remplacées par la médiane correspondante.

Trois modèles de classification ont ensuite été utilisés : la **régression logistique**, l'**arbre de décision** et la **Random Forest**. Leurs performances ont été évaluées à l'aide de l'Accuracy, de la Precision, du Recall et du F1-score.

Dans notre expérimentation, la Random Forest a obtenu les scores les plus élevés sur l'ensemble de test utilisé, avec une Accuracy de **77,92 %** et un F1-score de **65,31 %**.

Ce projet nous a permis de mettre en pratique les principales étapes d'un projet de Machine Learning : préparation des données, entraînement des modèles, prédiction, évaluation et comparaison des résultats. Il montre également l'importance de la qualité des données et du choix du modèle dans une tâche de classification.

7. Bibliographie

Sources principales

[1] Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C. & Johannes, R. S. (1988). *Using the ADAP learning algorithm to forecast the onset of diabetes mellitus*. Proceedings of the Symposium on Computer Applications in Medical Care, pp. 261–265.

[2] Kaggle. *Pima Indians Diabetes Database*. Dataset utilisé dans le cadre de ce projet. raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv

[3] Scikit-learn. *LogisticRegression – Documentation officielle*. Documentation de la bibliothèque utilisée pour la régression logistique. [Documentation LogisticRegression](#)

[4] Scikit-learn. *DecisionTreeClassifier – Documentation officielle*. Documentation de la bibliothèque utilisée pour l'arbre de décision. [Documentation DecisionTreeClassifier](#)

[5] Scikit-learn. *RandomForestClassifier – Documentation officielle*. Documentation de la bibliothèque utilisée pour la Random Forest. [Documentation RandomForestClassifier](#)

8. Annexes

Les annexes permettent de présenter les éléments techniques complémentaires sans alourdir les parties principales du rapport.

Et pour visualiser le projet complet avec ses codes vous pouvez regarder le fichier [projet_diabete.ipynb](#)

Annexe A – Code Python

Dans cette annexe, ce trouve quelques **codes utilisés dans Jupyter**.

A.1 Importation des bibliothèques

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

A.2 Chargement et exploration des données

```
df = pd.read_csv("diabetes.csv")

df.shape
df.info()
df.describe()
df.isnull().sum()
df["Outcome"].value_counts()
```

A.3 Identification des valeurs nulles

```
(df == 0).sum()
```

A.4 Nettoyage des données

```
df_clean = df.copy()
zero_as_missing = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]
df_clean[zero_as_missing] = df_clean[zero_as_missing].replace(0, np.nan)
for column in zero_as_missing:
    df_clean[column] = df_clean[column].fillna(
        df_clean[column].median()
    )
```

A.5 Préparation de X et y

```
X = df_clean.drop("Outcome", axis=1)
y = df_clean["Outcome"]
```

A.6 Séparation entraînement/test

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```